# Dart Web Build Commands Comparison

`dart run build_runner serve` vs `dart pub global run webdev build`

Understanding Modern Dart Web Development

# Overview

Two primary approaches for building Dart web applications:

🔧 **build_runner**: Modern, integrated development approach
🌐 **webdev**: Traditional, separate build tool approach

Both compile Dart to JavaScript, but with different workflows and capabilities.

## `dart run build_runner serve`

### What it does:

- **Serves** your application on a local development server
- **Watches** for file changes and rebuilds automatically
- **Hot reload** support for faster development
- **Integrated development** experience

### Requirements:

```
dev_dependencies:
  build_runner: ^2.3.3
  build_web_compilers: ^4.2.0
```

## `dart pub global run webdev build`

### What it does:

- **Builds** your application to static files
- **Outputs** compiled JavaScript to `build/` directory
- **One-time build** process (no serving)
- **Production-ready** output files

### Requirements:

```
dart pub global activate webdev
```

# Command Comparison Table

| Feature | build_runner serve | webdev build |
|---|---|---|
| **Purpose** | Development + Serving | Build Only |
| **Hot Reload** | ✅ Yes | ❌ No |
| **File Watching** | ✅ Automatic | ❌ Manual |
| **Output** | Memory + Server | Physical Files |
| **Production Ready** | ❌ Dev Only | ✅ Yes |

# Development Workflow

## build_runner serve

```
# Start development server
dart run build_runner serve
# ✅ Serves on http://localhost:8080
# ✅ Watches for changes
# ✅ Hot reload enabled
# ✅ Integrated debugging
```

## webdev build

```
# Build the project
dart pub global run webdev build
# ✅ Creates build/ directory
# ✅ Optimized JavaScript
# ❌ Need separate server for testing
```

# Performance Comparison

## Development Speed

- **build_runner serve**: ⚡ **Faster** (incremental compilation)
- **webdev build**: 🐌 **Slower** (full rebuild each time)

## Build Output

- **build_runner serve**: 💾 **Memory-based** (faster, no disk I/O)
- **webdev build**: 📁 **File-based** (persistent, shareable)

## Memory Usage

- **build_runner serve**: 🔋 **Higher** (keeps build in memory)
- **webdev build**: 🪫 **Lower** (builds and exits)

# Use Cases

## When to use `dart run build_runner serve`

- ✅ Active development
- ✅ Testing and debugging
- ✅ Rapid prototyping
- ✅ Learning and experimentation

## When to use `dart pub global run webdev build`

- ✅ Production deployment
- ✅ CI/CD pipelines
- ✅ Static hosting preparation
- ✅ Performance testing

# Setup Requirements

## build_runner approach

```
# pubspec.yaml
dev_dependencies:
  build_runner: ^2.3.3
  build_web_compilers: ^4.2.0
```

```
# build.yaml (optional)
targets:
  $default:
    builders:
      build_web_compilers:entrypoint:
        generate_for:
          - web/**.dart
```

# Setup Requirements (continued)

## webdev approach

```
# Global installation
dart pub global activate webdev

# Project dependencies
dart pub get

# Optional: local webdev
dart pub add --dev webdev
```

# File Structure Impact

## build_runner serve

```
project/
├── web/
│   ├── main.dart
│   └── index.html
├── lib/
├── pubspec.yaml
└── build.yaml
# No build/ directory during development
```

## webdev build

```
project/
├── web/
├── lib/
├── build/                  # ← Generated
│   └── web/
```

# Modern Recommendations

## For Students Learning

```
# Recommended: build_runner serve
dart run build_runner serve
```

**Why?** Immediate feedback, hot reload, integrated debugging

## For Production Deployment

```
# Recommended: webdev build
dart pub global run webdev build
```

**Why?** Optimized output, static files, hosting-ready

# Common Issues & Solutions

## build_runner serve

**Issue**: Port already in use

```
dart run build_runner serve --port 8081
```

**Issue**: Build cache problems

```
dart run build_runner clean
dart run build_runner serve
```

## webdev build

**Issue**: webdev not found

```
dart pub global activate webdev
```

# Advanced Usage

## build_runner serve with options

```
# Custom port
dart run build_runner serve --port 3000

# Release mode
dart run build_runner serve --release

# Specific directory
dart run build_runner serve web:8080
```

## webdev build with options

```
# Release mode (optimized)
dart pub global run webdev build --release

# Output directory
dart pub global run webdev build --output web:build
```

# Teaching Perspective

## For Classroom Use

**Recommend**: `dart run build_runner serve`

- Faster student feedback
- Less setup complexity
- Integrated development experience
- Better for live coding demos

## For Assignment Submission

**Recommend**: `dart pub global run webdev build`

- Creates shareable build artifacts
- Students learn production workflow

# Migration Path

## From webdev to build_runner

1. Remove global webdev dependency
2. Add build_runner to `dev_dependencies`
3. Create `build.yaml` configuration
4. Switch to `dart run build_runner serve`

## From build_runner to webdev

1. Install webdev globally
2. Remove build_runner dependencies
3. Use `webdev build` for production
4. Keep build_runner for development (optional)

# Summary

## Quick Decision Guide

### Choose `dart run build_runner serve` when:

- 🔄 Developing and testing
- 🚀 Need hot reload
- 📚 Teaching/learning
- ⚡ Want fastest development cycle

### Choose `dart pub global run webdev build` when:

- 🏭 Building for production
- 📦 Creating deployment artifacts
- 🤖 Setting up CI/CD
- 🌐 Preparing for static hosting

# Best Practices

## Development Workflow

```
# Daily development
dart run build_runner serve

# Before committing
dart run build_runner clean
dart pub global run webdev build --release
# Test the build/ output
```

## Project Setup

```
# Include both in scripts
scripts:
  dev: dart run build_runner serve
  build: dart pub global run webdev build --release
```

## Questions for Discussion

1. **Performance**: Which approach is faster for your development style?
2. **Deployment**: How does each approach fit your hosting strategy?
3. **Team Workflow**: Which is easier for collaborative development?
4. **Learning Curve**: Which is more appropriate for students?
5. **Maintenance**: How do dependency updates affect each approach?

# Thank You!

## Key Takeaways

- **build_runner serve**: Best for development
- **webdev build**: Best for production
- **Both are valid**: Choose based on your needs
- **Modern trend**: build_runner for integrated development

## Resources

- Dart Build System Documentation
- WebDev Tool Guide
- Dart Web Development Best Practices